

First Project

git version 2.50.1, Java 21, tomcat 9.0.122

DEPLOYING THE JAVA BASED APPLICATION USING JENKINS USING (FREE STYLE & PIPELINE)

- 1) Here firstly install git and java and Jenkins (web) in the server. Here we have taken t3micro with 8gb.
- 2) Now create another server and install the tomcat in it by using the script, so it becomes application server. (<https://dlcdn.apache.org/tomcat/tomcat-9/v9.0.122/bin/>) (username: tomcat and password: imjk04) and access with 8080 ports with server IP. And select the “manager app”
- 3) Now install the Jenkins by command and then start the Jenkins and then access the Jenkins dashboard.
- 4) After accessing the Jenkins dashboard, use the free style job pipeline.
- 5) In SCM Select the “git” and then select branch as the “master” and then now go to the tools in the manage Jenkins and then select the maven and give the name “mymaven”
- 6) In the build steps select the “Invoke top-level Maven targets” and specify the goal as the “clean install” and then click on build the project

```
root@ip-172-31-43-35:/var/lib/jenkins/workspace/Deployment
[root@ip-172-31-43-35 Deployment]# ll
total 12
-rw-r--r--. 1 jenkins jenkins 1723 Oct  8 11:34 pom.xml
-rw-r--r--. 1 jenkins jenkins  508 Oct  8 11:34 script.js
drwxr-xr-x. 3 jenkins jenkins   18 Oct  8 11:34 src
-rw-r--r--. 1 jenkins jenkins 2097 Oct  8 11:34 styles.css
drwxr-xr-x. 4 jenkins jenkins   70 Oct  8 11:34 target
[root@ip-172-31-43-35 Deployment]# pwd
/var/lib/jenkins/workspace/Deployment
[root@ip-172-31-43-35 Deployment]#
```

Here the source code is pulled to the Jenkins server and then build has happened so that's why the target folder is generated. In the target folder we have the “war” file.

- 7) Now install the plugin called “deploy to container” this plugin allows us to deploy the war file generated.
- 8) Now go to the Jenkins job and in the “post build actions” select the “deploy war/ear to container”
- 9) Complete Post Actions Setup, context path can be anything.

Deploy war/ear to a container

WAR/EAR files ?
target/*.war

Context path ?
e-commerce

Containers

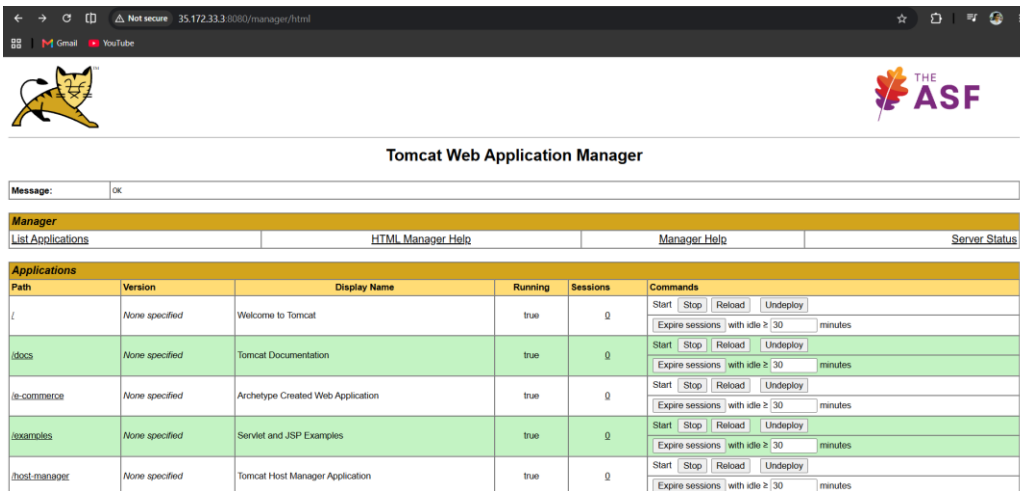
Tomcat 9.x Remote

Credentials
tomcat/* (tomcat-cred) + Add

Tomcat URL ?
http://35.172.33.3:8080/

Advanced

Tomcat Dashboard click on the /e-commerce which is context name to access the application



Till now we have code + build + unit_test + deploy.

1) Now we integrate the nexus, nexus is used to store the artifact. So that we have the different version of the war file, when we use the nexus, we cannot use the continuous deployment. Because we have to update the war file version in the pom.xml file.

2) Here we need the three servers, one for CI, Artifact, Deployment (Tomcat server), using the previous two server and now creating the nexus server with c7i, 15gb, connect the server and install the java.

3) Now go to the "self-hosted nexus page" and then copy the link for the linux using the command "wget" but download in "/opt" untar the file "tar -zxvf" and now in the same folder add the user called nexus.

4) change the owner of two folder sonar and nexus (chown -R nexus:nexus fd1 fd2) and switch to the nexus user

5) After switching to the nexus user, again go to "cd /opt" and go to nexus folder and start the nexus

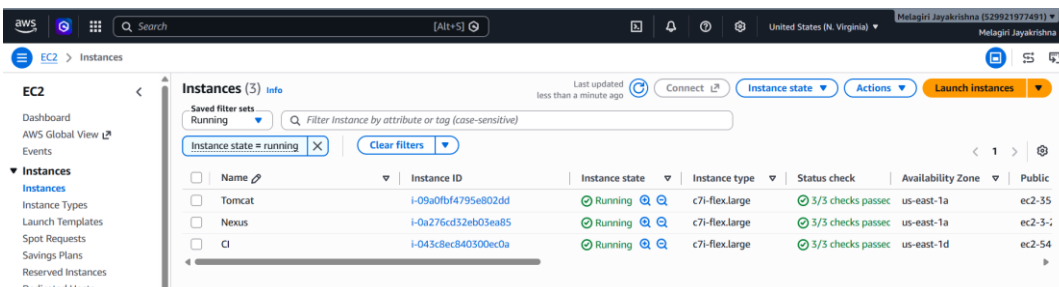
6) And access the nexus with the "8081" port and then "default username is admin and then get password from the path"

7) Now go to the setting and click on create repo and select the maven hosted and then "allow redploy"

8) Now integrate the nexus with jenkins, for this install the plugin called "nexus artifact uploader" add in the build steps after "invoke top level" and select "nexus artifact" and give the credentials

9) Give group id everything from the pom.xml file

10) Add the artifact dropdown and give the details



Build Steps

Automate your build process with ordered tasks like code compilation, testing, and deployment.

Invoke top-level Maven targets ?

Maven Version

mymaven

Goals

clean install

Advanced

Nexus artifact uploader

Nexus Details

Nexus Version

NEXUS3

Protocol

HTTP

GroupId

in.javahome

Nexus URL ?

3.231.58.99:8081

Version

8.7.3

Credentials

admin/***** (nexus-cred)

Repository ?

Nexus-Repo

Artifacts

Artifact

ArtifactId

myweb

Type ?

war

Classifier ?

File ?

target/myweb-8.7.3.war

Post-build Actions

Define what happens after a build completes, like sending notifications, archiving artifacts, or triggering other jobs.

Deploy war/ear to a container

WAR/EAR files ?

target/*.war

Context path ?

e-com

Containers

Tomcat 9.x Remote

Credentials

tomcat/***** (tomcat-cred)

+ Add

Tomcat URL ?

http://35.172.33.3:8080

The above is the build and post build actions.

sonatype nexus repository
Community Edition

Q Search components or CVEs...

Dashboard

Search

Browse

Upload

Malware Risk

Settings

Browse / Nexus-Repo

Upload component HTML View

Advanced search...

In

javahome

myweb

8.7.3

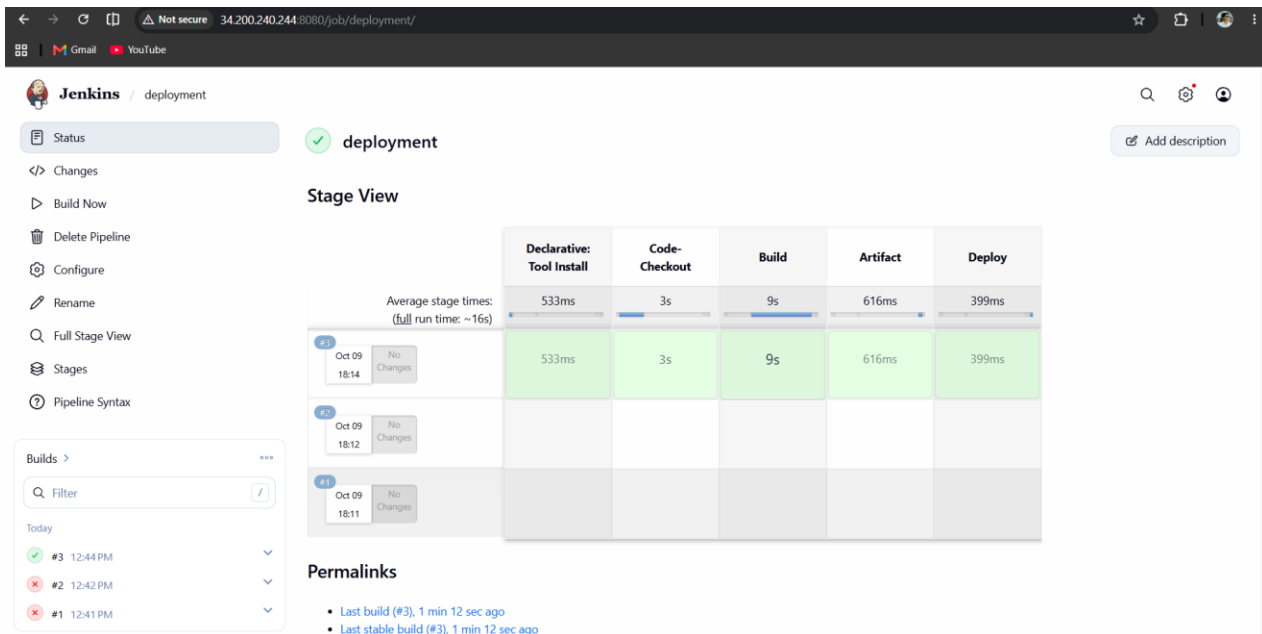
myweb-8.7.3.war

myweb-8.7.3.war.md5

myweb-8.7.3.war.sha1

NOW RUNNING THE PIPELINE USING THE SCRIPT

- 1) Perform as usual steps only. And declare the maven tool and install plugin called deploy to container and nexus artifact uploader.
- 2) In the script the select the pipeline script and then select the nexus artifact uploader, and give the all the details what we give for the nexus stage in the freestyle job



Till now we have run the pipeline in the Jenkins server itself, but now we use the agent to execute the pipeline and run the application in the agent itself (agent runs and execute the pipeline)

1) For this go to the Jenkins settings and then go to the nodes, and select add node and then use the private ip of the server where you want to execute the pipeline and select the ssh connection method and give the pem key for the authentication. Install the git in the tomcat server as well.

The screenshot shows the Jenkins 'Add Node' configuration page. The 'Launch method' is set to 'Launch agents via SSH'. The 'Host' is 172.31.10.211. The 'Credentials' are set to 'ec2-user'. The 'Host Key Verification Strategy' is 'Non verifying Verification Strategy'. The 'Availability' is 'Keep this agent online as much as possible'.

Plain text [Preview](#)

Number of executors [?](#)

2

Remote root directory [?](#)

/home/ec2-user/application

Labels [?](#)

dev

Usage [?](#)

Only build jobs with label expressions matching this node

Launch method [?](#)

Launch agents via SSH

Host [?](#)

172.31.10.211

Credentials [?](#)

ec2-user [+ Add](#)

Host Key Verification Strategy [?](#)

Non verifying Verification Strategy

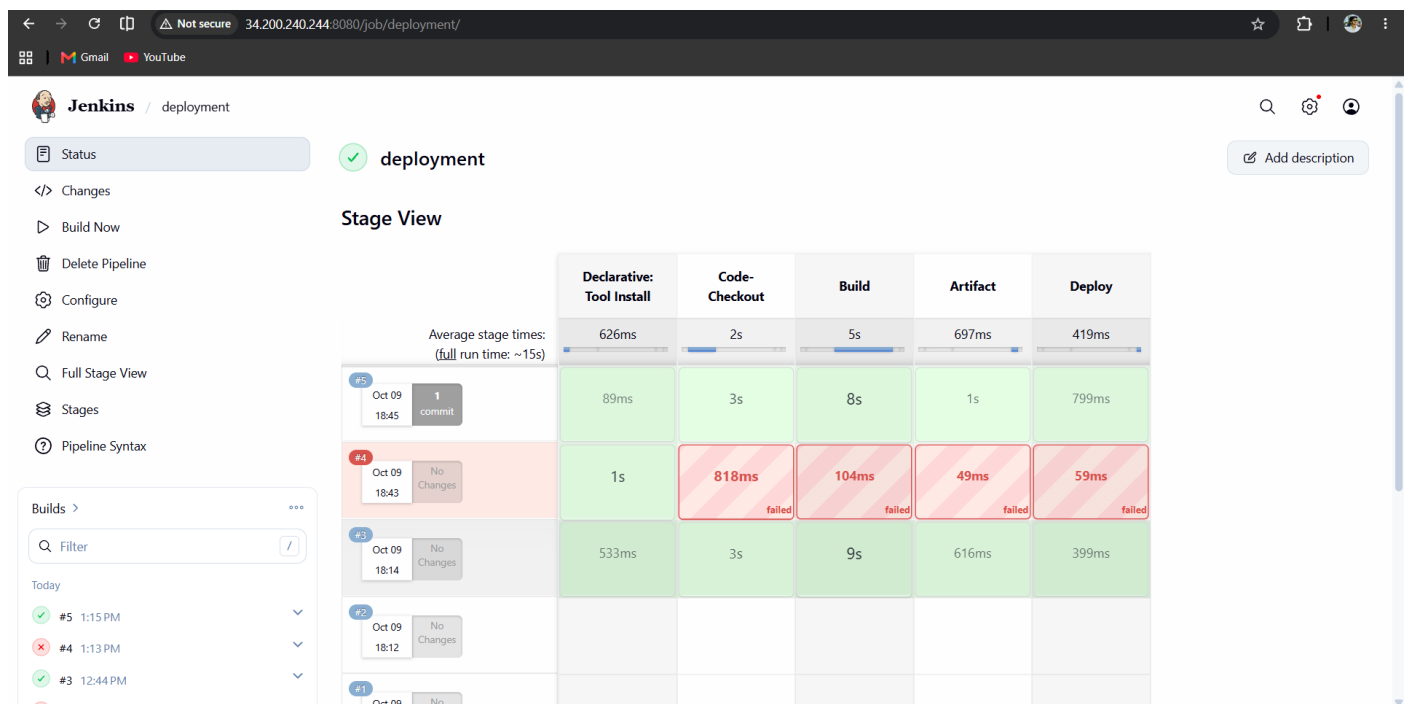
Advanced [v](#)

Availability [?](#)

Keep this agent online as much as possible

Now the pipeline is being executed in the application server itself (agent) and application is running in the same server as well. “Restrict where this project can be run” use this when you use free style job.

```
root@app-server/home/ec2-user/application/workspace/deployment
[root@app-server deployment]# ll
total 16
-rw-rw-r--. 1 ec2-user ec2-user 1001 Oct  9 13:15 pipeline
-rw-rw-r--. 1 ec2-user ec2-user 1723 Oct  9 13:15 pom.xml
-rw-rw-r--. 1 ec2-user ec2-user  508 Oct  9 13:15 script.js
drwxrwxr-x. 3 ec2-user ec2-user  18 Oct  9 13:15 src
-rw-rw-r--. 1 ec2-user ec2-user 2097 Oct  9 13:15 styles.css
drwxrwxr-x. 4 ec2-user ec2-user   70 Oct  9 13:15 target
[root@app-server deployment]# pwd
/home/ec2-user/application/workspace/deployment
[root@app-server deployment]#
```



Issue faced was

Built in node was offline, due to server configuration, fixed this by changing the server config.

Syntax issue, and then when we use controller and agent method did not install the git in the agent server dependency issue. So, pipeline failed.